Домашнее задание

Распечатать в PDF

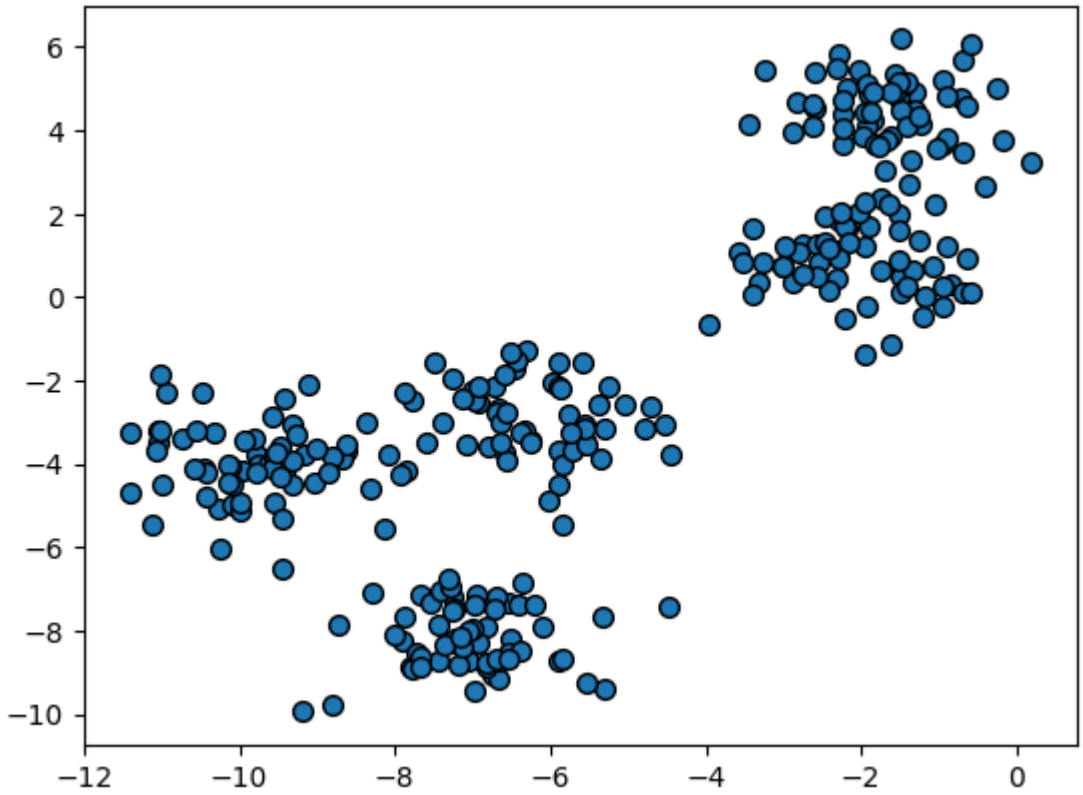
Содержание

- 1. Хотим пятерку, а на деле…
- 2. Повторение - мать ученья
- 3. Матрица расстояний (смежности) и матрица сходств
- 4. Игрушечные галактики (Источник - Pelican :))
- 5. *EM-алгоритм

1. Хотим пятерку, а на деле…

Кластеризуйте данный датасет с помощью метода `k-means`. На основе трёх различных **внутренних** метрик оценки кластеризации, подберите наилучшее количество кластеров k.

```
from sklearn.datasets import make_circles

X, y = make_blobs(n_samples=300, centers=5, cluster_std=0.85, random_stat
plt.scatter(X[:, 0], X[:, 1], s=50, edgecolor='k')
plt.show()
```



2. Повторение - мать ученья

Для предыдущего датасета подберите наилучшие гиперпараметры (мера несходства) с использованием агломеративного метода. Постройте дендрограмму для наилучшей модели.

3. Матрица расстояний (смежности) и матрица сходств

Расстояния между парами векторов из двух множеств $d\left(X_l, X_j\right)$ могут быть представлены в виде симметричной матрицы расстояний (матрица смежности):

$$D = \begin{pmatrix} 0 & d_{12} & \dots & d_{1n} \\ d_{21} & 0 & \dots & d_{2n} \\ \dots & \dots & \dots & \dots \\ d_{n1} & d_{n2} & \dots & 0 \end{pmatrix}$$

Понятием, противоположным расстоянию, является понятие сходства между объектами. Неотрицательная вещественная функция $S\left(x_i, x_j\right) = S_{ij}$ называется **мерой сходства**, если:

1. $0 \leq S\left(x_i, x_j\right) < 1$ для $x_i \neq x_j$
2. $S\left(x_i, x_i\right) = 1$
3. $S\left(x_i, x_j\right) = S\left(x_j, x_i\right)$

Пары значений мер сходства можно объединить в **матрицу сходства**:

$$S = \begin{pmatrix} 1 & s_{12} & \dots & s_{1n} \\ s_{21} & 1 & \dots & s_{2n} \\ s_{n1} & s_{n2} & \dots & 1 \end{pmatrix}$$

Величину S_{ij} называют **коэффициентом сходства**.

Постройте матрицу смежности по датасету первой задачи, взяв в качестве меры расстояния евклидову метрику.

Постройте на основе неё какую-нибудь матрицу сходства (вам надо самим придумать функцию сходства - просто удовлетвроите трём свойствам).

Визуализируйте обе матрицы в виде картинки. Используйте imshow() или pcolormesh() из Matplotlib.

Для получения матриц используйте функцию sklearn.metrics.pairwise_distances (тут можно в качестве метрики использовать свою собственную функцию - так можно сделать матрицу сходств) или какую-то другую отсюда https://scikit-learn.org/stable/modules/classes.html#module-sklearn.metrics.pairwise

Можно также посмотреть здесь https://scikit-learn.org/stable/modules/classes.html#module-sklearn.neighbors , например, функцию neighbors.kneighbors_graph

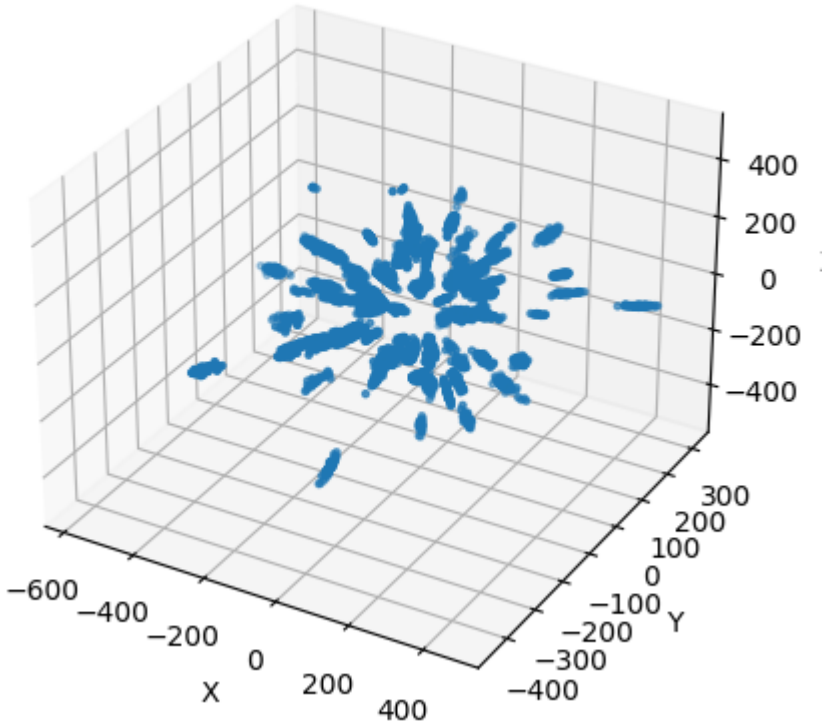
4. Игрушечные галактики (Источник - Pelican :))

Следующий датасет содержит координаты в 3д некоторого набора галактик. Необходимо найти скопления этих самых галактик с помощью алгоритма `hdbscan` - смеси обычного dbscan и иерархического подхода. Точная формулировка задачи после кода.

Вам понадобится файл toy_galaxies.csv из гугл диска.

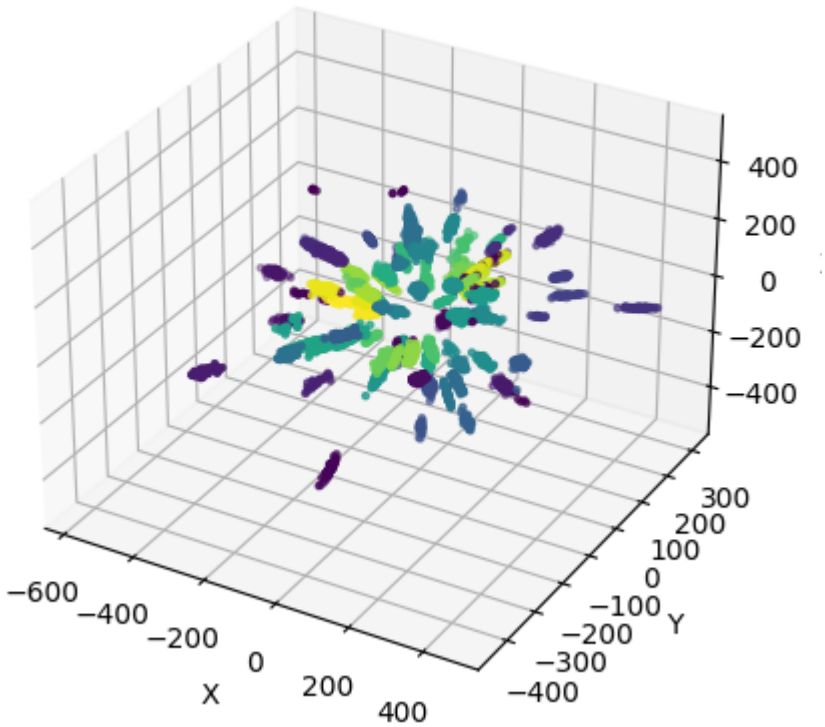
```
import numpy as np
from matplotlib import pyplot as plt
import pandas as pd
#!pip install hdbscan
import hdbscan
from sklearn.metrics import adjusted_rand_score as ari
from mpl_toolkits.mplot3d import Axes3D
from sklearn.cluster import MeanShift, AgglomerativeClustering


data = pd.read_csv("toy_galaxies.csv") # Не забудьте скачать файл
display(data.head(3))

# Отобразим датасет
fig = plt.figure(figsize=(5,5))
ax = fig.add_subplot(111, projection='3d')
Axes3D.scatter(ax, data.x,data.y,data.z, marker='.')
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')

plt.show()
```

	x	y	z	label
0	-35.283431	-21.779673	-113.964124	1.0
1	-35.603636	-18.430420	-113.298009	1.0
2	-34.202934	-22.362906	-106.979709	1.0



```
# Эта функция понадобится для отображения результата кластеризации и ARI
def plot_galaxies(data,cluster):
    fig = plt.figure(figsize=(5,5))

    ax = fig.add_subplot(111, projection='3d')
    Axes3D.scatter(ax,data.x, data.y, data.z, marker='.', c=cluster.labels
    ax.set_xlabel('X')
    ax.set_ylabel('Y')
    ax.set_zlabel('Z')
    plt.show();

    ARI = round(ari(data.label,cluster.labels_),4) # считаем ARI - оценка


    print('ARI = {}'.format(ARI))
```

```
# hdbscan работает следующим образом
cluster = hdbscan.HDBSCAN(metric="euclidean",
                          min_cluster_size=9,
                          algorithm="generic",
                          alpha=0.8,
                          cluster_selection_method='eom')

cluster.fit(data[['x','y','z']].to_numpy())

plot_galaxies(data,cluster) # Отображаем кластеризацию и полученный ARI
```

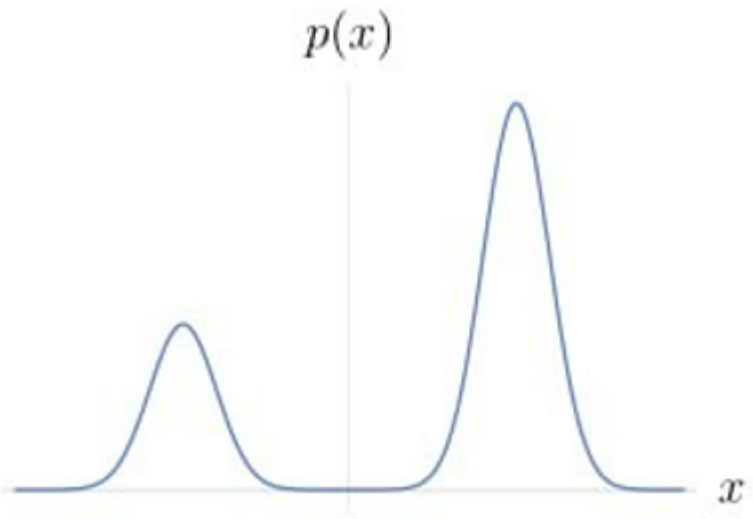



```
ARI = 0.8682
```

Ваша задача - перебирать различные параметры hdbscan (метрику, минимальный размер кластера и т.д. - надо залезть в документацию hdbscan и посмотреть, что там реализовано), чтобы получить `ARI 0.900` (а вообще, чем больше, тем лучше - удивите нас).

5. *EM-алгоритм

Задача матричного разложения (аппроксимация матрицы произведением двух других матриц меньшего ранга) с ограничениями (например, условие положительности элементов) не решается в общем случае с помощью сингулярного разложения. Для решения такой задачи может использоваться EM-алгоритм. Изучим его на примере другой простой модельной задачи.



Пусть дана выборка точек x_i, взятая из смеси гауссовых распределений:

$$p(x) = \alpha \cdot \mathcal{N}_{\mu_1,\sigma_1}(x) + (1-\alpha) \cdot \mathcal{N}_{\mu_2,\sigma_2}(x).$$

Тогда можно поставить задачу оценки параметров $\alpha, \mu_1, \mu_2, \sigma_1, \sigma_2$ по выборке $\{x_i\}$.

- Покажите, что задача максимизации обычного правдоподобия $\prod_i p\left(x_i\right) \longrightarrow \max_{\alpha,\mu_1,\mu_2}$ плохо определена. Какие значения параметров максимизируют такое правдоподобие?
- Сгенерируйте данные (просто два сгустка точек, хорошо видных при реализации) и найдите параметры $\alpha, \mu_1, \mu_2, \sigma_1, \sigma_2$ с помощью EM-алгоритма. Инициализировать параметры можно какими-то случайными значениями. EM-алгоритм состоит из двух чередующихся шагов:

1. M(Maximization)-шаг. Относим каждую точку x_i к первой или второй гауссиане, сравнивая значения правдоподобия для каждой компоненты смеси:

$$a\left(x_i\right)=\begin{cases}1, & p_1\left(x_i\right)>p_2\left(x_i\right),\\ 2, & p_2\left(x_i\right)>p_1\left(x_i\right),\end{cases}$$

где $p_1(x)=\alpha\mathcal{N}_{\mu_1,\sigma_1}(x),\ \ p_2(x)=(1-\alpha)\mathcal{N}_{\mu_2,\sigma_2}(x)$.

2. E(Expectation)-шаг. Находим параметры μ_1,σ_1 и μ_2,σ_2, максимизируя правдоподобие (или его логарифм) отдельно по точкам, отнесенным к каждой гауссиане:

$$\prod_{x_i:a(x_i)=1} p_1\left(x_i\right)\longrightarrow\max_{\mu_1,\sigma_1}$$
$$\prod_{x_i:a(x_i)=2} p_2\left(x_i\right)\longrightarrow\max_{\mu_2,\sigma_2}$$

Примечание. При нахождении параметра α можно оптимизировать обычное правдоподобие $\prod_i p\left(x_i\right)$. Все такие максимизации правдоподобия осуществляются аналитически в общем виде для гауссовых распределений.

Реализуйте EM-алгоритм. Так как метод является итерационным, необходимо выбрать какой-либо критерий остановки, например, прекращать процес, если относительное изменение каждого параметра при очередном шаге меньше некоторого порога. С какой точностью удалось восстановить $\alpha,\mu_1,\mu_2,\sigma_1,\sigma_2$?

Назад
Оценка качества кластеризации

Вперёд
2.5* Классификация. Нейронки